LFX Mentorship Fall 2025 — StackModule Submission

Fork URL: https://github.com/GreaseMonkeyIT/LFX-Mentorship

---

1. Source Code

src/main/scala/stack/StackModule.scala

```
package stack

import chisel3._
import chisel3.util._
import scala.math.min

class StackModule(val dataWidth: Int, val len: Int) extends Module {
  val io = IO(new Bundle {
    val in         = Input(UInt(32.W))
    val out        = Output(UInt(dataWidth.W))
    val underflow  = Output(Bool())
    val overflow   = Output(Bool())
    val isEmpty    = Output(Bool())
    val isFull     = Output(Bool())
    val popped     = Output(Bool())
    val peeked     = Output(Bool())
  })

  val opcode    = RegNext(io.in(6, 0), 0.U)
  val immediate = RegNext(io.in(31, 7), 0.U)

  // ===STATE===
  val sp    = RegInit(0.U(log2Ceil(len + 1).W))
  val stack = RegInit(VecInit(Seq.fill(len)(0.U(dataWidth.W))))

  // ===OUTPUT REGS===
  val outReg       = RegInit(0.U(dataWidth.W))
  val underflowReg = RegInit(false.B)
  val overflowReg  = RegInit(false.B)
  val poppedReg    = RegInit(false.B)
  val peekedReg    = RegInit(false.B)

  // ===FLAG CLEAR EACH CYCLE===
  outReg       := 0.U
  underflowReg := false.B
```

```
  overflowReg  := false.B
  poppedReg    := false.B
  peekedReg    := false.B

  // ===EXECUTE BASED ON REGISTERED OPCODE===
  switch(opcode) {
    is("b0100111".U) { // PUSH
      when(sp === len.U) {
        overflowReg := true.B
      }.otherwise {
        stack(sp) := immediate(min(dataWidth, 25) - 1, 0)
        sp := sp + 1.U
      }
    }
    is("b1000011".U) { // POP
      when(sp === 0.U) {
        underflowReg := true.B
      }.otherwise {
        sp := sp - 1.U
        outReg       := stack(sp - 1.U)
        poppedReg    := true.B
      }
    }
    is("b1000000".U) { // PEEK
      when(sp === 0.U) {
        underflowReg := true.B
      }.otherwise {
        outReg    := stack(sp - 1.U)
        peekedReg := true.B
      }
    }
  }

  // ===OUTPUT DRIVER===
  io.out       := outReg
  io.underflow := underflowReg
  io.overflow  := overflowReg
  io.popped    := poppedReg
  io.peeked    := peekedReg
  io.isEmpty   := sp === 0.U
  io.isFull    := sp === len.U
}
```

stack_tb.py

```python
from os import environ
from os.path import dirname, abspath, join
from random import randint

from cocotb import test, start_soon
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge
from bitstring import Bits

ROOT = dirname(abspath(__file__))


async def clear_stack(length, stack, dut, i, f):
    dut.reset.value = 1
    dut.io_in.value = 0

    await RisingEdge(dut.clock)   # 1: RESET DUT
    dut.reset.value = 0          # DEASSERT DUT RESET
    await RisingEdge(dut.clock)   # 2: DUT STATE UPDATE (CLEAR HAPPENS HERE)
    await RisingEdge(dut.clock)   # 3: OUTPUTS VALID

    stack.clear()
    assert dut.io_out.value       == 0
    assert dut.io_underflow.value == 0
    assert dut.io_overflow.value  == 0
    assert dut.io_isEmpty.value   == int(not stack)
    assert dut.io_isFull.value    == int(len(stack) == length)
    assert dut.io_popped.value    == 0
    assert dut.io_peeked.value    == 0

    f.write(f"Cycle: {i} | CLR         | Stack: {stack}\n")


async def push_to_stack(data_width, length, stack, dut, i, f):
    push_val = randint(0, 2 ** (data_width if data_width < 32 else 25) - 1)
    inst     = Bits(b32 = f'{push_val:025b}0100111')
    dut.io_in.value = inst.u
    dut.reset.value = 0

    await RisingEdge(dut.clock)   # 1: LATCH THE INSTRUCTION
    dut.io_in.value = 0           # CLEAR IT IMMEDIATELY
    await RisingEdge(dut.clock)   # 2: STATE UPDATE (PUSH HAPPENS HERE)
    await RisingEdge(dut.clock)   # 3: OUTPUTS VALID
```

```
    assert dut.io_out.value       == 0
    assert dut.io_underflow.value == 0
    assert dut.io_popped.value    == 0
    assert dut.io_peeked.value    == 0

    if len(stack) < length:
        stack.append(push_val)
        assert dut.io_overflow.value == 0
    else:
        assert dut.io_overflow.value == 1

    assert dut.io_isEmpty.value == int(not stack)
    assert dut.io_isFull.value  == int(len(stack) == length)

    f.write(f"Cycle: {i} | PUSH {push_val:>3}  | Stack: {stack}\n")


async def pop_from_stack(data_width, length, stack, dut, i, f):
    inst = Bits(b32='0'*25 + '1000011')
    dut.io_in.value = inst.u
    dut.reset.value = 0

    await RisingEdge(dut.clock)   # 1: LATCH THE INSTRUCTION
    dut.io_in.value = 0           # CLEAR IT IMMEDIATELY
    await RisingEdge(dut.clock)   # 2: STATE UPDATE (POP HAPPENS)
    await RisingEdge(dut.clock)   # 3: OUTPUTS VALID

    assert dut.io_overflow.value == 0
    assert dut.io_peeked.value   == 0

    if stack:
        expected = stack.pop()
        assert dut.io_underflow.value == 0
        assert dut.io_popped.value    == 1
        assert dut.io_out.value       == expected
    else:
        assert dut.io_underflow.value == 1
        assert dut.io_popped.value    == 0
        assert dut.io_out.value       == 0

    assert dut.io_isEmpty.value == int(not stack)
    assert dut.io_isFull.value  == int(len(stack) == length)

    f.write(f"Cycle: {i} | POP         | Stack: {stack}\n")
```

```python
async def peek_at_stack(data_width, length, stack, dut, i, f):
    inst = Bits(b32='0'*25 + '1000000')
    dut.io_in.value = inst.u
    dut.reset.value = 0

    await RisingEdge(dut.clock)   # 1: LATCH THE INSTRUCTION
    dut.io_in.value = 0           # CLEAR IT IMMEDIATELY
    await RisingEdge(dut.clock)   # 2: STATE UPDATE (PEEK HAPPENS)
    await RisingEdge(dut.clock)   # 3: OUTPUTS VALID

    assert dut.io_overflow.value == 0
    assert dut.io_popped.value   == 0

    if stack:
        expected = stack[-1]
        assert dut.io_underflow.value == 0
        assert dut.io_peeked.value    == 1
        assert dut.io_out.value       == expected
    else:
        assert dut.io_underflow.value == 1
        assert dut.io_peeked.value    == 0
        assert dut.io_out.value       == 0

    assert dut.io_isEmpty.value == int(not stack)
    assert dut.io_isFull.value  == int(len(stack) == length)

    f.write(f"Cycle: {i} | PEEK        | Stack: {stack}\n")


@test()
async def stack_tb(dut):
    start_soon(Clock(dut.clock, 1, units='ns').start(start_high=False))
    data_width = int(environ['DATA_WIDTH'])
    length     = int(environ['LENGTH'])
    stack      = []

    with open(join(ROOT, 'out', 'stack.SVGen', 'ref.log'), 'a') as f:
        for i in range(1_000):
            instID = randint(0, 3)
            if   instID == 0:
                await clear_stack(length, stack, dut, i, f)
            elif instID == 1:
                await push_to_stack(data_width, length, stack, dut, i, f)
            elif instID == 2:
                await pop_from_stack(data_width, length, stack, dut, i, f)
            else:
```

```
                await peek_at_stack(data_width, length, stack, dut, i, f)
```

---

2. Test Results

```
(.venv) dextopsd@LAPTOP-3UCL3DS6:~/RISC-V/Challenge/LFX-Mentorship$ make | tee sim_build/Vto
rm -f results.xml
"make" -f Makefile results.xml
make[1]: Entering directory '/home/dextopsd/RISC-V/Challenge/LFX-Mentorship'
rm -f results.xml
MODULE=stack_tb TESTCASE= TOPLEVEL=StackModule TOPLEVEL_LANG=verilog \
         sim_build/Vtop  --trace --trace-structs
     -.--ns INFO     gpi                                ..mbed/gpi_embed.cpp:108  in set_pro
     -.--ns INFO     gpi                                ../gpi/GpiCommon.cpp:101  in gpi_pri
     0.00ns INFO     cocotb                             Running on Verilator version 5.036 2
     0.00ns INFO     cocotb                             Running tests with cocotb v1.9.2 fro
     0.00ns INFO     cocotb                             Seeding Python random module with 17
     0.00ns INFO     cocotb.regression                  pytest not found, install it to enab
     0.00ns INFO     cocotb.regression                  Found test stack_tb.stack_tb
     0.00ns INFO     cocotb.regression                  running stack_tb (1/1)
  2999.50ns INFO     cocotb.regression                  stack_tb passed
                                                        ***********************************
                                                        ** TEST                          STA
                                                        ***********************************
                                                        ** stack_tb.stack_tb              PA
                                                        ***********************************
                                                        ** TESTS=1 PASS=1 FAIL=0 SKIP=0
                                                        ***********************************

- :0: Verilog $finish
stack_tb PASSED
```

Generated on: 2025-07-21 05:30